HDOC DAY-46

Title: Bubble Sort and First/Last Occurrence Search Using Variable-Length Arrays and Pointers

1. Problem Statement

Write a C program that reads the size and elements of a variable-length integer array, nums[]. The array may contain duplicate elements. Pass the array to a function named Bubble_sort() using its address and sort the elements in ascending order using the Bubble Sort technique. After sorting, pass the sorted array to another function named occurrence(). Use pointers to pass the array between functions. The function occurrence() must:

1. Read an integer target.
2. Find the first and last occurrences of target in the sorted array.
3. Display the values and indices of the first and last occurrences.
4. Display -1, -1 if the target is not present.

2. Theoretical Background & Algorithm Logic

- **Bubble Sort with Pointers:**

Iterates through adjacent elements using pointer arithmetic (*(arr + j)), comparing and swapping them until the entire array is sorted in ascending order.

- **Occurrence Search:**

Scans the sorted array linearly to locate the index where the target first appears (first_idx) and continuously updates last_idx until the last match is found. If no match occurs, it outputs -1, -1.

3. Test Case Table

Test Case	**Array Elements JPG+ 1**	**Target JPG + 1**	**Expected Output JPG+ 1**	**Actual Output**	**Status**
1	5 7 8 8 10	8	First occurrence: 8 at index 3 Last occurrence: 8 at index	First occurrence: 8 at index 3 Last occurrence: 8 at index 4	Pass

Test Case	Array Elements JPG+ 1	Target JPG + 1	Expected Output JPG+ 1	Actual Output	Status
			4[cite: 31]		
2	5 7 8 8 10 [cite: 31, 32]	6	-1, -1 [cite: 32]	-1, -1	Pass

4. Source Code Listing

```
#include <stdio.h>

void Bubble_sort(int *arr, int size) {
   for (int i = 0; i < size - 1; i++) {
      for (int j = 0; j < size - i - 1; j++) {
         if (*(arr + j) > *(arr + j + 1)) {
            int temp = *(arr + j);
            *(arr + j) = *(arr + j + 1);
            *(arr + j + 1) = temp;
         }
      }
   }
}

void occurrence(int *arr, int size) {
   int target;
   printf("Enter integer target: ");
   scanf("%d", &target);
```

```
    int first_idx = -1, last_idx = -1;
    for (int i = 0; i < size; i++) {
        if (*(arr + i) == target) {
            if (first_idx == -1) first_idx = i;
            last_idx = i;
        }
    }

    if (first_idx != -1) {
        printf("First occurrence: %d at index %d\n", target, first_idx);
        printf("Last occurrence: %d at index %d\n", target, last_idx);
    } else {
        printf("-1, -1\n");
    }
}

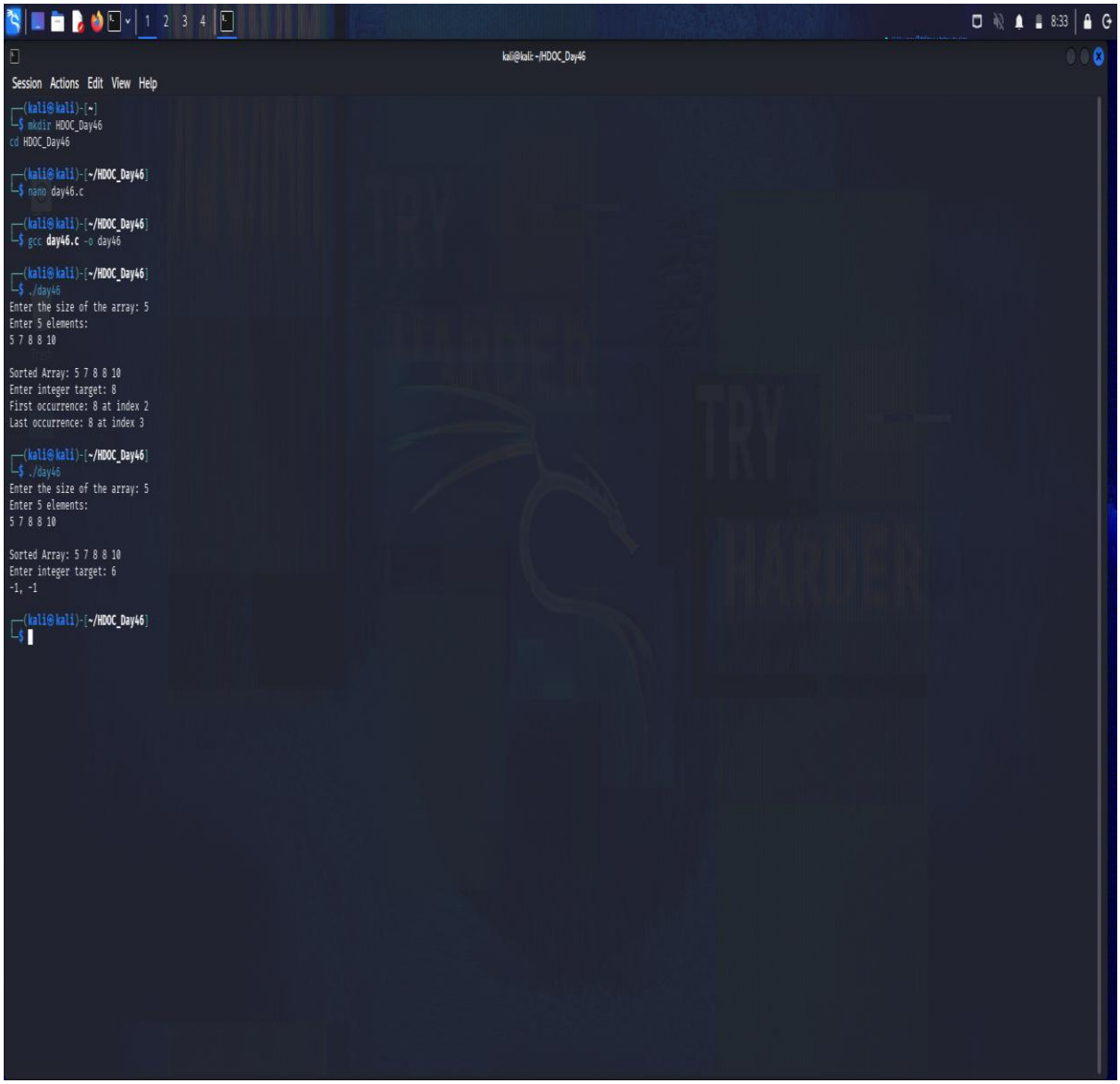
int main(void) {
    int size;
    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size <= 0) return 1;

    int nums[size];
    printf("Enter %d elements:\n", size);
    for (int i = 0; i < size; i++) scanf("%d", &nums[i]);

    Bubble_sort(nums, size);

    printf("Sorted Array: ");
    for (int i = 0; i < size; i++) printf("%d ", nums[i]);
    printf("\n");

    occurrence(nums, size);
    return 0;
}
```


kali@kali: ~/HDOC_Day46
Session Actions Edit View Help
(kali㉿kali)-[~]
$ mkdir HDOC_Day46
cd HDOC_Day46
(kali㉿kali)-[~/HDOC_Day46]
$ nano day46.c
(kali㉿kali)-[~/HDOC_Day46]
$ gcc day46.c -o day46
(kali㉿kali)-[~/HDOC_Day46]
$ ./day46
Enter the size of the array: 5
Enter 5 elements:
5 7 8 8 10
Sorted Array: 5 7 8 8 10
Enter integer target: 8
First occurrence: 8 at index 2
Last occurrence: 8 at index 3
(kali㉿kali)-[~/HDOC_Day46]
$ ./day46
Enter the size of the array: 5
Enter 5 elements:
5 7 8 8 10
Sorted Array: 5 7 8 8 10
Enter integer target: 6
-1, -1
(kali㉿kali)-[~/HDOC_Day46]
$